

MMJL portable smoke test

This notebook tests the drop-in `sys.path` setup, `register_jupy_logger(root=...)`, quiet magic wrappers, pin-family behavior, relative manifest paths, relative timeline links, validation, and HTML/Markdown export.

It is intended to run in:

- local JupyterLab
- VS Code notebooks
- AWS SageMaker Jupyter spaces

Expected source layouts:

- local repo: `./src/multimodal_jupy_logger`
- SageMaker/drop-in: `~/multimodal-jupy-logger/src`

```
In [1]: from pathlib import Path
import importlib
import os
import shutil
import sys

USE_EXPLICIT_MMJL_SRC = True

EXPLICIT_MMJL_SRC = Path(
    r"D:\David\my_repos_dwb\multimodal-jupy-logger\src"
)

starting_dir = Path.cwd().resolve()

candidate_src_paths = []

if USE_EXPLICIT_MMJL_SRC:
    candidate_src_paths.append(
        EXPLICIT_MMJL_SRC.expanduser().resolve()
    )
```

```
else:
    for candidate in [starting_dir, *starting_dir.parents]:
        candidate_src_paths.append(candidate / "src")
    ##endof: for candidate in [...]

    candidate_src_paths.append(
        Path.home() / "multimodal-jupy-logger" / "src"
    )
##endof: if USE_EXPLICIT_MMJL_SRC

mmjl_src = None

for candidate_src in candidate_src_paths:
    if (candidate_src / "multimodal_jupy_logger").is_dir():
        mmjl_src = candidate_src
        break
    ##endof: if (candidate_src / ...)
##endof: for candidate_src in candidate_src_paths

if mmjl_src is None:
    checked_paths = "\n".join(
        f" - {candidate}"
        for candidate in candidate_src_paths
    )

    raise RuntimeError(
        "Could not find MMJL source.\n"
        "Checked:\n"
        f"{checked_paths}"
    )
##endof: if mmjl_src is None

if str(mmjl_src) not in sys.path:
    sys.path.insert(0, str(mmjl_src))
##endof: if str(mmjl_src) not in sys.path

importlib.invalidate_caches()

print("source mode:", (
    "explicit"
    if USE_EXPLICIT_MMJL_SRC
    else "portable discovery"
```

```

))
print("starting_dir:", starting_dir)
print("mmjl_src:", mmjl_src)
print("python:", sys.executable)

```

source mode: explicit

starting_dir: D:\David\tmp\try_1

mmjl_src: D:\David\my_repos_dwb\multimodal-jupy-logger\src

python: D:\David\my_repos_dwb\multimodal-jupy-logger\.venv_mmjl_test\Scripts\python.exe

```

In [2]: from pathlib import Path
import shutil

from multimodal_jupy_logger import (
    MultimodalJupyLogger,
    register_jupy_logger,
)

log_root = Path.cwd() / "jupy_log_smoke_test"

# Smoke test root is intentionally disposable.
if log_root.exists():
    shutil.rmtree(log_root)
##endof: if log_root.exists()

register_jupy_logger(root=log_root)

print("MMJL log root:", log_root)
print("log root exists:", log_root.exists())

```

[DONE] Registered: %jupy_save, %jupy_file, %%jupy_log, %%jupy_capture, %%jupy_tee, %jupy_markdown, %jupy_html, %jupy_inspect, %jupy_validate

MMJL log root: D:\David\tmp\try_1\jupy_log_smoke_test

log root exists: True

```

In [3]: %jupy_inspect

```

```
Manifest: D:\David\tmp\try_1\jupy_log_smoke_test\manifest.tsv
Root: D:\David\tmp\try_1\jupy_log_smoke_test
Artifacts: D:\David\tmp\try_1\jupy_log_smoke_test\artifacts
Staging: D:\David\tmp\try_1\jupy_log_smoke_test\staging
Timelines: D:\David\tmp\try_1\jupy_log_smoke_test\timelines
Rows: 0
```

By kind:

By MIME:

Literal logging smoke test

Expected:

- no `WindowsPath(...)` display after the magic
- one `[DONE] Logged text: ...` status line
- a manifest row with a relative `artifacts/...` path

```
In [4]: %%jupy_log --label smoke-markdown-note --mime text/markdown
# Smoke Markdown note
```

This text was logged with `%%jupy\_log`.

It should appear in the manifest and exported timelines.

```
[DONE] Logged text: D:\David\tmp\try_1\jupy_log_smoke_test\artifacts\000001_1783856703497_2026-07-12T074503497-0400_s
smoke-markdown-note.md
```

Pin family smoke tests

Expected:

- `--pin` logs input + stdout with an automatic label
- `--label NAME` with no pin option behaves like `--pin`
- `--pin-input` logs input only
- `--pin-output` logs output only

```
In [5]: %%jupy_capture --pin
        pin_message = "pin captures input and output"
        print(pin_message)
```

pin captures input and output
[DONE] Capture 000001: logged 2 artifact(s).

```
In [6]: %%jupy_capture --label named-pin-default
        named_message = "label alone behaves like pin"
        print(named_message)
```

label alone behaves like pin
[DONE] Capture 000002: logged 2 artifact(s).

```
In [7]: %%jupy_capture --label input-only-example --pin-input
        quiet_value = sum([1, 2, 3, 4])
        print("This output should show in Jupyter, but not be logged by --pin-input.")
```

This output should show in Jupyter, but not be logged by --pin-input.
[DONE] Capture 000003: logged 1 artifact(s).

```
In [8]: %%jupy_capture --label output-only-example --pin-output
        output_only_value = 6 * 7
        print("This output should be logged, but the input should not be logged.")
        output_only_value
```

This output should be logged, but the input should not be logged.
42
[DONE] Capture 000004: logged 2 artifact(s).

Plot capture smoke test

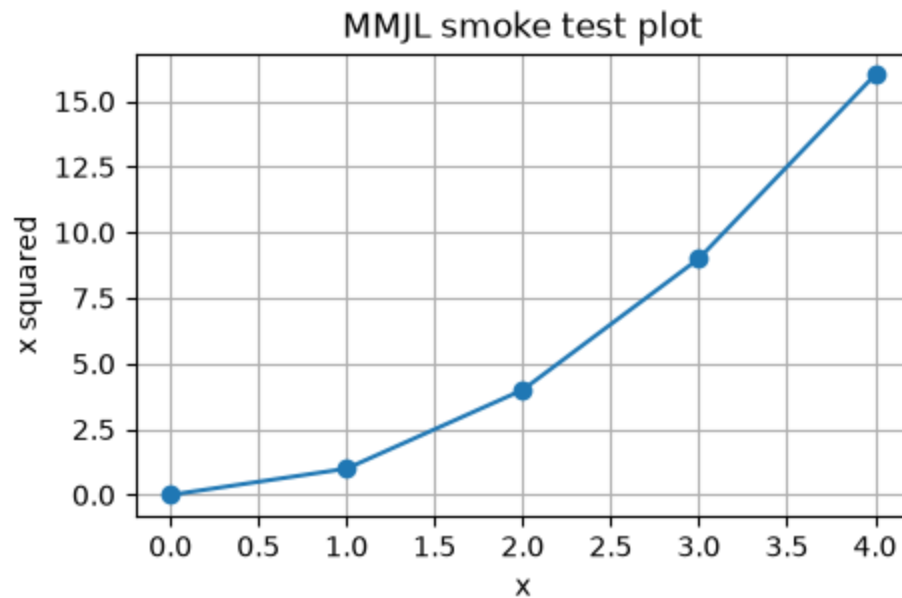
Expected:

- stdout is logged
- Matplotlib display is logged as an image artifact
- timeline HTML/Markdown uses relative links to that image

```
In [9]: %%jupy_capture --label tiny-plot --pin
        import matplotlib.pyplot as plt
```

```
xs = [0, 1, 2, 3, 4]
ys = [x * x for x in xs]

plt.figure(figsize=(5, 3))
plt.plot(xs, ys, marker="o")
plt.title("MMJL smoke test plot")
plt.xlabel("x")
plt.ylabel("x squared")
plt.grid(True)
plt.show()
```



[DONE] Capture 000005: logged 3 artifact(s).

Exception capture smoke test

Expected:

- Jupyter shows an error
- MMJL logs the input and exception artifact
- continuing with later cells still works after manually running the next cell

Run this cell intentionally. The error is expected.

```
In [10]: %%jupy_capture --label expected-exception --pin
print("This cell intentionally raises an exception.")
raise ValueError("Expected MMJL smoke-test exception")
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[11], line 2
      1 print("This cell intentionally raises an exception.")
----> 2 raise ValueError("Expected MMJL smoke-test exception")

ValueError: Expected MMJL smoke-test exception
This cell intentionally raises an exception.
[DONE] Capture 000006: logged 3 artifact(s).
[MMJL] %%jupy_capture captured an execution error.
```

Validate and export

Expected:

- `%jupy_validate` reports zero missing artifacts
- `%jupy_markdown` and `%jupy_html` create timeline files
- no `[]` or `WindowsPath(...)` appears as extra notebook output from the magic wrappers

```
In [11]: %jupy_inspect
          %jupy_validate
          %jupy_markdown
          %jupy_html
```

Manifest: D:\David\tmp\try_1\jupy_log_smoke_test\manifest.tsv
Root: D:\David\tmp\try_1\jupy_log_smoke_test
Artifacts: D:\David\tmp\try_1\jupy_log_smoke_test\artifacts
Staging: D:\David\tmp\try_1\jupy_log_smoke_test\staging
Timelines: D:\David\tmp\try_1\jupy_log_smoke_test\timelines
Rows: 14

By kind:

image: 1
text: 13

By MIME:

image/png: 1
text/markdown: 1
text/plain: 7
text/x-python: 5

Manifest: D:\David\tmp\try_1\jupy_log_smoke_test\manifest.tsv

Rows checked: 14

Missing artifacts: 0

[DONE] Markdown timeline written: D:\David\tmp\try_1\jupy_log_smoke_test\timelines\timeline.md

[DONE] HTML timeline written: D:\David\tmp\try_1\jupy_log_smoke_test\timelines\timeline.html

```
In [12]: from pathlib import Path
import csv

manifest_path = log_root / "manifest.tsv"
timeline_md = log_root / "timelines" / "timeline.md"
timeline_html = log_root / "timelines" / "timeline.html"

print("manifest:", manifest_path)
print("timeline_md:", timeline_md)
print("timeline_html:", timeline_html)

assert manifest_path.exists()
assert timeline_md.exists()
assert timeline_html.exists()

rows = list(csv.DictReader(
    manifest_path.open("r", encoding="utf-8", newline=""),
    delimiter="\t",
))
```



```

print("manifest rows:", len(rows))

absolute_paths = [
    row["path"]
    for row in rows
    if Path(row["path"]).is_absolute()
]

print("absolute manifest paths:", absolute_paths)

assert not absolute_paths, "Manifest should use relative artifact paths."

md_text = timeline_md.read_text(encoding="utf-8")
html_text = timeline_html.read_text(encoding="utf-8")

assert "../artifacts/" in md_text or "../artifacts/" in html_text
assert str(log_root.resolve()) not in md_text
assert str(log_root.resolve()) not in html_text

print("[PASS] Portable relative-path smoke test passed.")

```

```

manifest: D:\David\tmp\try_1\jupy_log_smoke_test\manifest.tsv
timeline_md: D:\David\tmp\try_1\jupy_log_smoke_test\timelines\timeline.md
timeline_html: D:\David\tmp\try_1\jupy_log_smoke_test\timelines\timeline.html
manifest rows: 14
absolute manifest paths: []
[PASS] Portable relative-path smoke test passed.

```

Manual checks

After the test, inspect:

```

jupy_log_smoke_test/
├── artifacts/
├── staging/
├── timelines/
│   ├── timeline.html
│   └── timeline.md
└── manifest.tsv

```

The manifest `path` column should contain paths like:

artifacts/000001_..._smoke-markdown-note.md

The timeline files should link to artifacts with:

../artifacts/...

```
In [13]: import os
import shutil
import subprocess

EXCLUDED_TREE_TEXT = (
    "ipynb_checkpoints",
    "checkpoint.ipynb",
    "serial",
)

def _filter_tree_output(output: str) -> str:
    kept_lines = (
        line
        for line in output.splitlines()
        if not any(
            excluded in line
            for excluded in EXCLUDED_TREE_TEXT
        )
    )

    return "\n".join(kept_lines)
##endof: _filter_tree_output

def _run_tree_command(command: list[str]) -> str:
    result = subprocess.run(
        command,
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="replace",
        check=True,
    )
```

```
        return _filter_tree_output(result.stdout)
    ##endof: _run_tree_command

def _run_windows_tree() -> str:
    if os.name != "nt":
        raise RuntimeError(
            "The Windows tree command requires Windows."
        )

    tree_command = shutil.which("tree")

    if tree_command is None:
        raise RuntimeError(
            "The Windows 'tree' command was not found."
        )

    return _run_tree_command(
        [
            tree_command,
            "/a",
            "/f",
            ".",
        ]
    )
##endof: _run_windows_tree

def _run_linux_tree() -> str:
    if os.name == "nt":
        raise RuntimeError(
            "The Linux tree command requires a non-Windows system."
        )

    tree_command = shutil.which("tree")

    if tree_command is None:
        raise RuntimeError(
            "The Linux 'tree' command was not found."
        )

    return _run_tree_command(
```

```

        [
            tree_command,
            "-a",
            "-f",
            ".",
        ]
    )
##endof: _run_linux_tree

def get_filtered_tree() -> str:
    windows_error: Exception | None = None
    linux_error: Exception | None = None

    try:
        return _run_windows_tree()
    except (
        RuntimeError,
        subprocess.CalledProcessError,
    ) as error:
        windows_error = error

    try:
        return _run_linux_tree()
    except (
        RuntimeError,
        subprocess.CalledProcessError,
    ) as error:
        linux_error = error

    raise RuntimeError(
        "Only trying Windows and Linux tree commands.\n"
        f"Windows attempt: {windows_error}\n"
        f"Linux attempt: {linux_error}"
    ) from linux_error
##endof: get_filtered_tree

try:
    tree_output = get_filtered_tree()
    print(tree_output)
except RuntimeError as error:

```

```

    print(error)
    raise
finally:
    pass
##endof: try/except/finally

```

Folder PATH listing for volume Data

D:\DAVID\TMP\TRY_1

| mmjl_portable_smoke_test.ipynb

|

\---jupy_log_smoke_test

| manifest.tsv

|

+---artifacts

| 000001_1783856703497_2026-07-12T074503497-0400_smoke-markdown-note.md

| 000002_1783862106474_2026-07-12T091506474-0400_pin-000001-input.py

| 000003_1783862106492_2026-07-12T091506492-0400_pin-000001-stdout.txt

| 000004_1783862108645_2026-07-12T091508645-0400_named-pin-default-input.py

| 000005_1783862108669_2026-07-12T091508669-0400_named-pin-default-stdout.txt

| 000006_1783862112120_2026-07-12T091512120-0400_input-only-example-input.py

| 000007_1783862114129_2026-07-12T091514129-0400_output-only-example-stdout.txt

| 000008_1783862114145_2026-07-12T091514145-0400_output-only-example-display-01-01-text.txt

| 000009_1783862177883_2026-07-12T0916177883-0400_tiny-plot-input.py

| 000010_1783862181071_2026-07-12T091621071-0400_tiny-plot-display-01-01-text.txt

| 000011_1783862181085_2026-07-12T091621085-0400_tiny-plot-display-01-02-image.png

| 000012_1783862221580_2026-07-12T091701580-0400_expected-exception-input.py

| 000013_1783862222184_2026-07-12T091702184-0400_expected-exception-stdout.txt

| 000014_1783862222228_2026-07-12T091702228-0400_expected-exception-exception.txt

+---staging

\---timelines

| timeline.html

| timeline.md

In []: